# GPUMODE Lecture Study Notes: Lecture 26 SYCL Mode

## Core Problem the Lecture Addresses

This lecture explains how to program and optimize accelerators with **SYCL**, especially Intel GPUs. The motivation is that much GPU education is CUDA centered, while real heterogeneous systems may include CPUs, Intel GPUs, AMD GPUs, NVIDIA GPUs, FPGAs, and other accelerators. SYCL gives a C++ programming model for these devices, but the programmer still needs hardware knowledge to get good performance.

The main problem is not only how to launch a kernel. The deeper problem is how to reason about data movement, work mapping, memory hierarchy, execution resources, and profiling. The lecture shows that performance often depends less on peak arithmetic throughput and more on whether the program avoids unnecessary memory traffic.

> SYCL improves portability, but performance still comes from understanding the hardware target.

## Required Background

### Heterogeneous Computing

Heterogeneous computing means using multiple kinds of processors in one system. The CPU usually handles control flow, setup, I/O, and orchestration. The GPU handles computational hotspots where many independent operations can run in parallel.

A typical heterogeneous program has the following shape.

```
int main() {
    // CPU: allocate and initialize data.
    // CPU: select a device.
    // CPU to GPU: move input data.
    // GPU: run a parallel kernel.
    // GPU to CPU: move result data back.
```

```
    // CPU: validate or continue the application.
}
```

The CPU and GPU may have separate memory spaces. If they are separate devices, data may cross an interconnect such as PCIe. This is much slower than accessing data already on the GPU.

### Why Low-Level Programming Matters

High-level tools are useful, but efficient high-level code still depends on low-level facts. An engineer should understand device memory, host memory, queues, work-items, work-groups, local memory, cache behavior, SIMD execution, occupancy, and memory bandwidth.

A compact performance model is

fast accelerator code = parallelism+efficient memory movement+hardware-aware scheduling.

## Main Concepts

### What SYCL Provides

SYCL is a modern C++ programming model for heterogeneous systems. A basic SYCL program needs three things:

1. A way to select and query a hardware device.
2. A way to allocate and move data.
3. A way to express parallel work.

This can be summarized as

$$\text{SYCL program} = \text{device abstraction} + \text{memory model} + \text{parallel kernel}.$$

### Devices and Queues

A **device** represents a CPU, GPU, FPGA, or another accelerator. A **queue** is the object through which the host submits work to the device.

```
#include <sycl/sycl.hpp>
#include <iostream>

using namespace sycl;

int main() {
    queue q(gpu_selector{});

    std::cout << "Running on: "
```

```
              << q.get_device().get_info<info::device::name>()
              << std::endl;

    return 0;
}
```

The queue is the command path from the CPU to the GPU. It can contain memory copies, kernel launches, and synchronization operations. Many queue operations are asynchronous. The host continues unless the program explicitly waits.

```
q.submit([&](handler& h) {
    h.parallel_for(range<1>(N), [=](id<1> i) {
        data[i] = data[i] * 2.0f;
    });
});

q.wait();
```

The call `q.wait()` synchronizes the host with queued device work.

## Memory Allocation and Movement

SYCL supports explicit device memory and unified shared memory. Explicit memory management gives the programmer more control. Shared memory is convenient, but it may hide migration costs.

```
float* h = malloc_host<float>(N, q);
float* d = malloc_device<float>(N, q);
float* s = malloc_shared<float>(N, q);
```

The meanings are:

- `malloc_host`: host-accessible allocation.
- `malloc_device`: device allocation.
- `malloc_shared`: memory accessible from host and device.

An explicit copy flow is shown below.

```
constexpr int N = 1024;
queue q(gpu_selector{});

float* host_data = malloc_host<float>(N, q);
float* device_data = malloc_device<float>(N, q);

for (int i = 0; i < N; ++i) {
    host_data[i] = static_cast<float>(i);
}
```

```
q.memcpy(device_data, host_data, N * sizeof(float)).wait();

q.submit([&](handler& h) {
    h.parallel_for(range<1>(N), [=](id<1> idx) {
        int i = idx[0];
        device_data[i] = 2.0f * device_data[i];
    });
}).wait();

q.memcpy(host_data, device_data, N * sizeof(float)).wait();
```

This kernel is memory-bound. Each element performs about one floating-point multiply and moves one input and one output. For FP32, the approximate arithmetic intensity is

$$I \approx \frac{1}{8} \text{ FLOPs per byte.}$$

## Parallel Kernels

SYCL expresses independent parallel work with `parallel_for`. A sequential loop such as

```
for (int i = 0; i < N; ++i) {
    a[i] = b[i] + c[i];
}
```

can be written as

```
q.submit([&](handler& h) {
    h.parallel_for(range<1>(N), [=](id<1> idx) {
        int i = idx[0];
        a[i] = b[i] + c[i];
    });
}).wait();
```

This is correct only if loop iterations are independent. A simple condition is

$$\text{writes}(i) \cap \text{writes}(j) = \varnothing \quad \text{for} \quad i \neq j.$$

## Work-Items, Work-Groups, and Ranges

A **work-item** is the basic logical unit of parallel execution. It is similar to a CUDA thread. A **work-group** is a group of work-items. It is similar to a CUDA thread block.

For a one-dimensional launch,

$$\text{global range} = \text{number of groups} \times \text{local range}.$$

With global size $N$ and local size $L$,

$$G = \frac{N}{L}.$$

For $N = 1024$ and $L = 64$,

$$G = 16.$$

Explicit control uses `nd_range`.

```
constexpr int N = 1024;
constexpr int local_size = 64;

q.submit([&](handler& h) {
    h.parallel_for(
        nd_range<1>(range<1>(N), range<1>(local_size)),
        [=](nd_item<1> item) {
            int gid = item.get_global_id(0);
            int lid = item.get_local_id(0);
            int group = item.get_group(0);

            data[gid] = data[gid] + 1.0f;
        });
}).wait();
```

The indexing relation is

$$gid = group \cdot L + lid.$$

# Hardware-Level Explanation

## Intel GPU Execution Hierarchy

The lecture describes Intel GPU architecture with a hierarchy like

$$\text{GPU} \to \text{slices} \to \text{subslices} \to \text{execution units} \to \text{SIMD lanes}.$$

An Intel subslice is roughly comparable to an SM-like region in NVIDIA terminology. An execution unit, abbreviated EU, is where instructions are executed. EUs contain arithmetic resources and register state for multiple execution contexts.

This supports latency hiding. If one context waits for memory, another context can execute. The simplified idea is

$$\text{many resident contexts} + \text{fast switching} \Rightarrow \text{latency hiding}.$$

### SIMD Execution and Work Size

If the SIMD width is 8, one instruction operates on eight data elements. If a work-group exposes too little work, many lanes are idle. With only one active element on an eight-lane unit,

$$\text{lane utilization} = \frac{1}{8} = 12.5\%.$$

This explains why tiny work-groups can perform badly. The GPU needs enough work-items and enough work-groups to fill execution resources and hide latency.

### Memory Hierarchy

A simplified memory hierarchy is

$$\text{registers} \rightarrow \text{shared local memory} \rightarrow \text{L1 cache} \rightarrow \text{L3 cache} \rightarrow \text{global memory}.$$

Closer memory is faster but smaller. Farther memory is larger but slower. The goal is to load data from global memory as few times as possible and then reuse it from registers or shared local memory.

#### Visual Concept

Draw global memory at the bottom, then L3 cache, then several subslices. Inside one subslice, draw shared local memory, L1 cache, execution units, SIMD lanes, and register states. Show data moving upward toward execution units and being reused locally.

## Mathematical Reasoning

### Arithmetic Intensity and the Roofline Model

Arithmetic intensity is

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

A roofline-style upper bound is

$$P \leq \min\left(P_{\text{peak}},\ IB_{\text{mem}}\right),$$

where $P$ is achievable performance, $P_{\text{peak}}$ is peak compute throughput, and $B_{\text{mem}}$ is memory bandwidth.

If

$$IB_{\text{mem}} < P_{\text{peak}},$$

then the kernel is memory-bound. If

$$P_{\text{peak}} < IB_{\text{mem}},$$

then the kernel is compute-bound.

## Matrix Multiplication

For matrix multiplication,

$$C = AB,$$

where

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

Each output element is

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}.$$

The sequential code is

```
for (int i = 0; i < M; ++i) {
    for (int j = 0; j < N; ++j) {
        float acc = 0.0f;
        for (int k = 0; k < K; ++k) {
            acc += A[i * K + k] * B[k * N + j];
        }
        C[i * N + j] = acc;
    }
}
```

The outer loops over $i$ and $j$ are independent. The inner loop over $k$ is a reduction into one accumulator. It cannot be naively parallelized without a reduction strategy.

The total operation count is

$$\text{FLOPs} = 2MNK.$$

## Naive Parallel GEMM

A simple GPU mapping assigns one work-item to one output element:

$$\text{work-item}(i, j) \to C_{i,j}.$$

```
q.submit([&](handler& h) {
    h.parallel_for(
        nd_range<2>(
            range<2>(M, N),
            range<2>(block_size, block_size)),
        [=](nd_item<2> item) {
            int row = item.get_global_id(0);
            int col = item.get_global_id(1);

            float acc = 0.0f;
```

```
            for (int k = 0; k < K; ++k) {
                acc += A[row * K + k] * B[k * N + col];
            }

            C[row * N + col] = acc;
        });
}).wait();
```

This is easy to understand but not optimal, because neighboring work-items repeatedly reload overlapping data from global memory.

## DLRM Interaction Layer

The case study is a DLRM-style interaction layer. It contains operations similar to

$$\text{concat} \rightarrow \text{BMM} \rightarrow \text{index} \rightarrow \text{concat}.$$

Let batch size be $B = 32768$, number of feature vectors be $E = 27$, and embedding dimension be $D = 128$. The number of unique off-diagonal feature pairs is

$$\frac{E(E-1)}{2} = \frac{27 \cdot 26}{2} = 351.$$

Each dot product over the embedding dimension performs approximately $2D$ FLOPs. Therefore, the interaction FLOP count is

$$\text{FLOPs} = B \cdot \frac{E(E-1)}{2} \cdot 2D.$$

Substituting the values gives

$$\text{FLOPs} = 32768 \cdot 351 \cdot 256.$$

The lecture's key observation is that the surrounding operations are mostly memory movement. Therefore, the end-to-end layer can be memory-bound even though BMM performs arithmetic.

## FP16 and Memory Traffic

For a tensor with $N_{\text{elem}}$ elements,

$$\text{bytes}_{\text{FP32}} = 4N_{\text{elem}}, \quad \text{bytes}_{\text{FP16}} = 2N_{\text{elem}}.$$

Thus,

$$\frac{\text{bytes}_{\text{FP16}}}{\text{bytes}_{\text{FP32}}} = \frac{1}{2}.$$

In a memory-bound kernel, FP16 can improve performance because it halves memory traffic.

# Code Walkthrough

## Tiled GEMM With Local Memory

The lecture motivates shared local memory through profiling. A representative tiled GEMM pattern is shown below.

```
constexpr int TILE = 16;

q.submit([&](handler& h) {
    local_accessor<float, 2> Asub(range<2>(TILE, TILE), h);
    local_accessor<float, 2> Bsub(range<2>(TILE, TILE), h);

    h.parallel_for(
        nd_range<2>(
            range<2>(M, N),
            range<2>(TILE, TILE)),
        [=](nd_item<2> item) {
            int row = item.get_global_id(0);
            int col = item.get_global_id(1);
            int local_row = item.get_local_id(0);
            int local_col = item.get_local_id(1);

            float acc = 0.0f;

            for (int t = 0; t < K; t += TILE) {
                Asub[local_row][local_col] =
                    A[row * K + (t + local_col)];

                Bsub[local_row][local_col] =
                    B[(t + local_row) * N + col];

                item.barrier(access::fence_space::local_space);

                for (int k = 0; k < TILE; ++k) {
                    acc += Asub[local_row][k] * Bsub[k][local_col];
                }

                item.barrier(access::fence_space::local_space);
            }

            C[row * N + col] = acc;
        });
}).wait();
```

The hardware idea is

load once from global memory → reuse many times from local memory.

The barriers are required because all work-items in the work-group must finish loading the tile before any work-item consumes it.

### Fused DLRM Interaction Pseudocode

A fused interaction kernel avoids materializing intermediate concat and full BMM tensors.

```
q.submit([&](handler& h) {
    h.parallel_for(
        nd_range<2>(
            range<2>(batch_size, num_pairs),
            range<2>(1, local_pairs)),
        [=](nd_item<2> item) {
            int b = item.get_global_id(0);
            int pair_id = item.get_global_id(1);

            int i = pair_i[pair_id];
            int j = pair_j[pair_id];

            float acc = 0.0f;

            for (int d = 0; d < embedding_dim; ++d) {
                float xi = X[(b * num_features + i) * embedding_dim + d];
                float xj = X[(b * num_features + j) * embedding_dim + d];
                acc += xi * xj;
            }

            Y[b * num_pairs + pair_id] = acc;
        });
}).wait();
```

This directly computes only the needed interaction values. The avoided memory traffic is approximately

$$\Delta\text{bytes} \approx \text{bytes written for intermediates} + \text{bytes read for intermediates}.$$

## Performance Intuition

### Memory-Bound Versus Compute-Bound Behavior

A kernel is memory-bound when runtime is dominated by moving data. A kernel is compute-bound when runtime is dominated by arithmetic. The DLRM interaction case is mostly memory-bound because concat, index, and intermediate materialization move large tensors.

The practical question is not only how many FLOPs the kernel performs. The better question is how many bytes are moved per useful result.

## Why Tensor Cores or DPAS May Not Help

Specialized matrix instructions improve compute throughput. They do not remove memory traffic. If

$$T_{\text{memory}} \gg T_{\text{compute}},$$

then reducing compute time gives limited end-to-end speedup.

For example,

$$T_{\text{old}} = 1.0\ \mu\text{s} + 0.1\ \mu\text{s} = 1.1\ \mu\text{s}.$$

If compute becomes ten times faster,

$$T_{\text{new}} = 1.0\ \mu\text{s} + 0.01\ \mu\text{s} = 1.01\ \mu\text{s}.$$

The speedup is only

$$\frac{1.1}{1.01} \approx 1.09.$$

## Why Shared Local Memory Helps

If profiler data shows repeated L3 reads, the kernel has locality but is reusing data from a relatively distant cache. Moving reusable data into shared local memory can reduce repeated L3 traffic.

The before pattern is

$$\text{global memory} \rightarrow \text{L3} \rightarrow \text{execution units repeatedly.}$$

The improved pattern is

$$\text{global memory} \rightarrow \text{L3} \rightarrow \text{shared local memory} \rightarrow \text{execution units repeatedly.}$$

## Kernel Fusion

Kernel fusion combines multiple operators into one kernel. For an unfused sequence,

$$Y = f(X), \quad Z = g(Y),$$

the memory pattern is read $X$, write $Y$, read $Y$, and write $Z$. A fused kernel computes

$$Z = g(f(X))$$

without writing $Y$ to global memory.

This is especially valuable when $Y$ is large and the computation is bandwidth-bound.

## Common Misconceptions

- **Misconception: SYCL is only CUDA with different syntax.** SYCL is a cross-platform C++ model, while CUDA is primarily NVIDIA-specific.
- **Misconception: the GPU should run the whole program.** Usually only the parallel hotspot belongs on the GPU.
- **Misconception: any loop can become a parallel loop.** Only independent iterations can be directly parallelized.
- **Misconception: tensor cores always solve performance.** They help compute-bound kernels, not memory-bound kernels.
- **Misconception: FP16 helps only through faster arithmetic.** In many kernels it helps by reducing memory traffic.
- **Misconception: cache hits mean the kernel is optimal.** Repeated L3 access can still be worse than shared local memory reuse.

## Practical Takeaways

1. SYCL programs are built from devices, queues, memory management, and parallel kernels.
2. Queues are asynchronous, so synchronization must be used intentionally.
3. Work-items are similar to CUDA threads, and work-groups are similar to CUDA thread blocks.
4. Work-group size affects SIMD utilization, occupancy, and latency hiding.
5. Naive GEMM parallelizes over output elements but does not reuse memory efficiently.
6. Tiling improves reuse by staging data into shared local memory.
7. FP16 can help memory-bound kernels by halving bytes moved.
8. Kernel fusion can outperform separate library calls when intermediate memory traffic dominates.
9. Profiling should guide optimization decisions instead of guessing.

## Project or Experiment Ideas

### Experiment 1: Memory Management

Implement vector addition using explicit device memory and then using shared memory. Compare runtime and programming complexity.

### Experiment 2: Work-Group Sweep

Sweep local sizes 1, 8, 16, 32, 64, 128, and 256 for a vector scaling kernel. Measure throughput and identify when increasing the work-group size stops helping.

### Experiment 3: GEMM Variants

Compare naive GEMM against tiled GEMM. Compute achieved throughput with

$$\text{GFLOPs} = \frac{2MNK}{T \cdot 10^9}.$$

### Experiment 4: FP32 Versus FP16

Run a memory-bound kernel in FP32 and FP16. If memory bandwidth dominates, the expected relationship is

$$T_{\text{FP16}} < T_{\text{FP32}}.$$

### Experiment 5: DLRM Interaction Fusion

Implement an unfused concat, BMM, index, concat pipeline. Then implement a fused pairwise interaction kernel. Measure allocated memory, global memory traffic, and runtime.

## Final Mental Model

SYCL gives a portable C++ interface for heterogeneous programming, but performance still comes from GPU systems thinking. A work-item maps to hardware execution resources. A work-group controls how work is organized. Data moves through registers, local memory, caches, and global memory. Each unnecessary intermediate tensor creates extra reads and writes.

The final model is

$$\boxed{\text{high performance} = \text{good work mapping} + \text{low memory traffic} + \text{local reuse} + \text{profiling feedback}}.$$

For the DLRM case study, the important optimization was not simply faster BMM. The important optimization was reducing memory movement through fusion, FP16 storage, and shared local memory reuse.